SKILL - 10

Name: k.vaishali

Rollno: 2520030117

Aim

To retrieve the current directory, display the path, process exit requests, clean up resources, save required state, and verify correct behavior. Also, to parse export syntax, update environment variables, validate variable names, handle existing variables, export variables to child processes, and test the complete functionality.

Objective

- To retrieve and display the current working directory.
- To process exit requests correctly.
- To clean up allocated resources before termination.
- To save and maintain required program state.
- To verify correct shell behavior.
- To parse export command syntax.
- To validate environment variable names.

Problem Statement

Design and implement a shell-style program that can display the current working directory, process an exit request, maintain and save required state, and clean up resources before termination. The program should also parse export commands, validate variable names, create or update environment variables, make exported variables available to child processes, and verify the complete behavior.

Theory

1. Current Directory

The current working directory identifies the directory in which the shell process is currently operating. The getcwd() function retrieves this path so it can be displayed to the user.

2. Display Path

Displaying the current path helps the user understand the shell's location. A built-in pwd command commonly provides this functionality without creating a separate child process.

3. Exit Requests

An exit request tells the shell to stop accepting input. Before terminating, the program should save required state and release allocated resources.

4. Resource Cleanup

Resources such as dynamically allocated memory must be released before termination. Centralized cleanup helps prevent memory leaks and ensures controlled exit.

5. Saving State

Shell state may include variables or other information needed during the session. Required state can be preserved before the command interpreter exits.

6. Export Syntax

The export command associates a variable name with a value and makes it part of the environment inherited by child processes.

7. Environment Variables

Environment variables store named values used by programs and child processes. setenv() updates an environment value and getenv() retrieves it.

8. Variable Name Validation

A variable name must begin with a letter or underscore and may contain letters, digits, and underscores. Invalid names are rejected before the environment changes.

9. Existing Variables

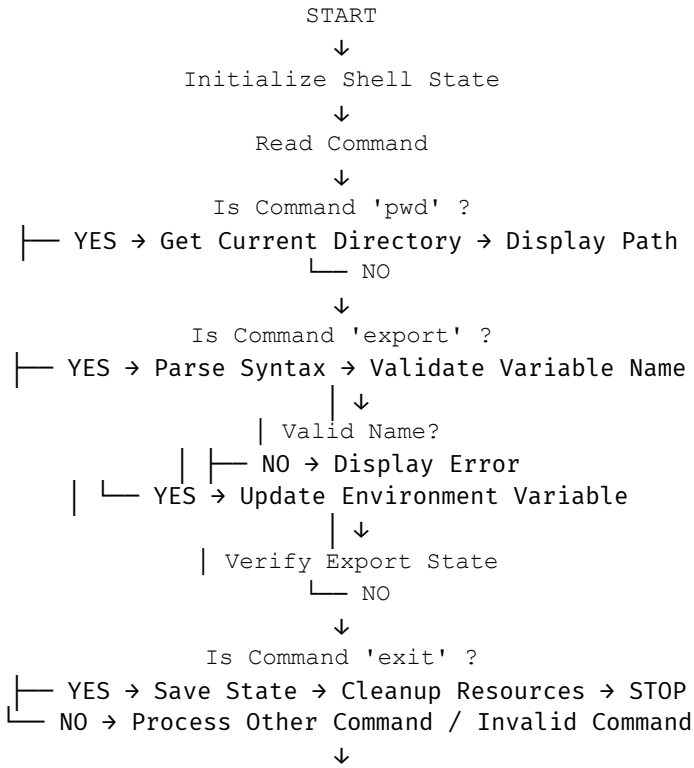
When export refers to an existing variable, setenv() updates the value using overwrite

behavior. **10. Child Processes**

Exported environment variables are inherited by child processes created after the variable is added to the environment.

11. Control Flow

The complete control flow is kept on this page so that the flowchart is not divided across pages.



12. Algorithm

1. Start the program.
2. Initialize the shell and required state variables.
3. Display the command prompt.
4. Read a command from the user.
5. If the command is pwd, retrieve and display the current working directory.
6. If the command begins with export, parse the variable name and value.
7. Validate the variable name using the required naming rules.
8. If the name is invalid, display an error message.
9. If the name is valid, update the environment variable using setenv().
10. Verify that the variable is available through getenv().
11. When a child process is launched after export, allow it to inherit the environment.
12. If the command is exit, save required state and clean up resources.
13. Terminate the shell after successful cleanup.
14. For other commands, display an appropriate response.
15. Repeat the command loop until exit is requested.
16. Stop the program.

13. Data Used

Character arrays are used for commands, variable names, values, and paths. The environment is maintained by the operating system through setenv() and getenv(). Integer status values control command execution and exit behavior.

14. Operations Used

- getcwd() for retrieving the current directory.
- strtok() and string checks for parsing export syntax.
- Character checks for variable-name validation.
- setenv() for creating or updating environment variables.
- getenv() for verifying environment values.
- fork() and exec*() for testing inheritance in a child process.
- waitpid() for parent-child synchronization.
- Memory/resource cleanup before exit.

15. Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>
#include <limits.h>

#define MAX 512

int valid_name(const char *name) {
 if (name == NULL || name[0] == '\0') return 0;
 if (!(isalpha((unsigned char)name[0]) || name[0] == '_')) return 0;
 for (int i = 1; name[i] != '\0'; i++)
 if (!(isalnum((unsigned char)name[i]) || name[i] == '_')) return 0;
 return 1;
}
```

```
void show_path() {
 char path[PATH_MAX];
 if (getcwd(path, sizeof(path)) != NULL) printf("%s\n", path);
 else perror("getcwd");
 }

void verify_child(const char *name) {
 pid_t pid = fork();
 if (pid < 0) { perror("fork"); return; }

 if (pid == 0) {
 char *value = getenv(name);
 printf("Child process:\n");
 if (value != NULL) printf("%s=%s\n", name, value);
else printf("%s is not available\n", name);
 exit(0);
 }
 waitpid(pid, NULL, 0);
 }

int main() {
 char input[MAX];

 printf("Skill - 10: Directory, Exit and Environment Management\n");
printf("-----------------------------------------------------------\n");

 while (1) {
 printf("\nskill10> ");
 fflush(stdout);

 if (fgets(input, sizeof(input), stdin) == NULL) break;
input[strcspn(input, "\n")] = '\0';
 if (strlen(input) == 0) continue;

 if (strcmp(input, "pwd") == 0) {
 printf("Current directory:\n");
 show_path();
 }
 else if (strncmp(input, "export ", 7) == 0) {
 char command[MAX], *name, *value;
 strcpy(command, input + 7);

 name = strtok(command, "=");
 value = strtok(NULL, "");
 if (value == NULL) value = "";

 if (!valid_name(name)) {
 printf("Invalid variable name: %s\n", name); continue;
 }

 if (setenv(name, value, 1) == 0) {
 printf("Export successful: %s=%s\n", name, value); printf("Verified
with getenv: %s=%s\n", name, getenv(name)); verify_child(name); }
else {
 perror("setenv");
 }
 }
 else if (strcmp(input, "exit") == 0) {
 printf("Saving shell state...\n");
 printf("Cleaning up resources...\n");
 printf("Exiting Skill-10 shell...\n");
 break;
 }
 else {
 printf("Invalid command: %s\n", input);
 }
 }

 return 0;
 }
```

16. Program Explanation

16.1 Current Directory

The pwd section uses getcwd() to retrieve and display the current directory directly from the operating

system. **16.2 Export Parsing**

The export section separates the variable name from its value using '=' and validates the name before changing the environment.

16.3 Variable Name Validation

valid_name() checks the first character and then every remaining character to reject invalid

identifiers. **16.4 Updating Environment**

setenv() creates a new variable or replaces an existing value. getenv() verifies the

result. **16.5 Child Process Export**

After export, fork() creates a child process and the child calls getenv() to confirm

inheritance. **16.6 Exit and Cleanup**

The exit branch represents controlled termination and reports state saving and resource cleanup before leaving the command loop.

17. Sample Output

```
Skill - 10: Directory, Exit and Environment Management
-------------------------------------------------------

skill10> pwd
Current directory:
/Users/Hansika/Downloads

skill10> export COURSE=DSA
Export successful: COURSE=DSA
Verified with getenv: COURSE=DSA
Child process:
COURSE=DSA

skill10> export COURSE=OperatingSystems
Export successful: COURSE=OperatingSystems
Verified with getenv: COURSE=OperatingSystems
Child process:
COURSE=OperatingSystems

skill10> export 1COURSE=DSA
Invalid variable name: 1COURSE

skill10> exit
Saving shell state...
Cleaning up resources...
Exiting Skill-10 shell…
```

18. Output Screenshot

Paste the screenshot of the program output in the space provided below.

```
./skill10
Skill - 10: Directory, Exit and Environment Management
------------------------------------------------------

skill10> pwd
export COURSE=DSA
export COURSE=OperatingSystems
export 1COURSE=DSA
export STUDENT=Hansika
exitCurrent directory:
/Users/7rainreddy7

skill10> Export successful: COURSE=DSA
Verified with getenv: COURSE=DSA
Child process:
COURSE=DSA

skill10> Export successful: COURSE=OperatingSystems
Verified with getenv: COURSE=OperatingSystems
Child process:
COURSE=OperatingSystems

skill10> Invalid variable name: 1COURSE
```

19. Testing

- Tested retrieval of the current working directory.
- Tested a valid export command.
- Tested updating an existing environment variable.
- Tested an invalid variable name.
- Verified the updated variable with getenv().
- Verified that the exported variable was available in a child process.
- Tested the exit request and cleanup sequence.

20. Test Cases

Test Case	Input Expected Result
1	pwd Current working directory is displayed.
2	export COURSE=DSA Variable is created and exported.
3	export COURSE=OS Existing variable is updated.
4	export 1COURSE=DSA Invalid variable name is rejected.
5	getenv in child process Child receives the exported variable.
6	exit State is saved, cleanup is performed, and the shell exits.

21. Observation

- The current working directory was retrieved and displayed correctly.
- Export syntax was parsed using the variable name and assigned value.
- Invalid variable names were rejected.
- New environment variables were created successfully.
- Existing variables were updated successfully.
- getenv() verified the current environment value.
- The exported variable was inherited by the child process.
- The exit request followed the cleanup path correctly.

22. Result

The current directory retrieval, exit handling, resource cleanup, state management, and environment-variable export functionality were successfully implemented. Export syntax was parsed and validated, existing variables were updated, and exported variables were available to child processes.

23. Conclusion

The practical demonstrated how a shell can maintain execution state and environment information while providing built-in commands for directory display, controlled exit, and environment management. The implementation verified variable validation, environment updates, child-process inheritance, and orderly termination.